Reviewer Pack — Morse Theory Invariant for DPLL Search Trees

Entry 46001aee0695 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 02:28:30 UTC

Morse Theory Invariant for DPLL Search Trees

Entry ID: `46001aee0695` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-31 02:28:30 UTC

1. Conjecture

Field A (mathematical branch): Morse Theory in Differential Geometry **Field B** (complexity object): DPLL search trees in complexity theory

Statement:

> For a given Boolean formula φ with n variables, the number of critical points of the Morse function associated with the DPLL search tree of φ is upper bounded by O(n^(2/3)).

Rationale (proposer's reasoning):

> Morse Theory provides a geometric approach to classify critical points in manifolds, which can be applied to study the structure of the DPLL search tree. Since the size of a DPLL tree grows exponentially with the number of variables, understanding the distribution of its critical points could provide insights into the complexity of satisfiability problems.

Taxonomy category: `MORSE_THEORY_DPLL` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2a6f7bc7a91d7c40`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, across all 30 seeds, the mean number of critical points in the Morse function of DPLL search trees for Boolean formulas φ with n variables is less than or equal to O(n^(2/3)) AND the correlation coefficient between the number of critical points and n^(2/3) is above 0.9.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Morse Theory" AND "DPLL search trees" - "critical points" IN "Morse function" AND "Boolean formula" - "complexity theory" AND "differential geometry"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(n):
        if n == 1:
            return 'A'
        else:
            subformulas = [generate_formula(n-1) for _ in range(2)]
            return f'({subformulas[0]} & {subformulas[1]})'

    def dpll_search_tree(formula, assignment):
        if formula == 'A':
            return 1
        elif formula.startswith('('):
            left, right = formula[1:-1].split(' & ')
            return dpll_search_tree(left, assignment) + dpll_search_tree(right, assignment)
        else:
            var = formula
            if var in assignment:
                return 0
            else:
                true_assignment = assignment.copy()
                true_assignment[var] = True
                false_assignment = assignment.copy()
                false_assignment[var] = False
                return dpll_search_tree(formula, true_assignment) + dpll_search_tree(formula, fals

    def morse_function(n):
        formula = generate_formula(n)
        assignment = {}
        return dpll_search_tree(formula, assignment)
```

```
    n_values = [5, 10, 15, 20, 30, 40]
    critical_points = []

    for n in n_values:
        for _ in range(5):  # Ensure at least 30 instances per seed
            cp = morse_function(n)
            if cp is not None:
                critical_points.append(cp)

    if len(critical_points) == 0:
        return {
            "metric_name": "Critical Points",
            "metric_value": 0,
            "instances_tested": 0,
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": "No critical points found"
        }

    mean_cp = sum(critical_points) / len(critical_points)
    n_cubed_root = [n ** (2/3) for n in n_values]
    correlation_sum = sum((cp - mean_cp) * (n_val ** (2/3) - mean(n_cubed_root)) for cp, n_val in
    variance_n_cubed_root = sum((n_val ** (2/3) - mean(n_cubed_root)) ** 2 for n_val in n_values)

    if variance_n_cubed_root == 0:
        return {
            "metric_name": "Critical Points",
            "metric_value": mean_cp,
            "instances_tested": len(critical_points),
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": "Variance of n^(2/3) is zero"
        }

    correlation_coefficient = correlation_sum / (len(critical_points) * math.sqrt(variance_n_cubed

    return {
        "metric_name": "Critical Points",
        "metric_value": mean_cp,
        "instances_tested": len(critical_points),
        "n_max": max(n_values),
        "conjecture_holds": correlation_coefficient >= 0.9,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
```

```
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_09de1f7f.py", line 101, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_09de1f7f.py", line 55, in run_trial
    cp = morse_function(n)
         ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_09de1f7f.py", line 48, in morse_functio
    return dpll_search_tree(formula, assignment)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_09de1f7f.py", line 32, in dpll_search_t
    left, right = formula[1:-1].split(' & ')
    ^^^^^^^^^^^
ValueError: too many values to unpack (expected 2)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means we cannot verify the conjecture's support conditions. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	23993
2	propose	ollama_remote	glm4:latest	0	0	14514
3	preregistration	ollama_remote	glm4:latest	0	0	9773
4	novelty	ollama_remote	glm4:latest	0	0	15632
5	novelty	ollama_remote	glm4:latest	0	0	9275
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17556
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15532
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11372
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11882
10	judge	ollama_remote	glm4:latest	0	0	21056

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 150585 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/46001aee0695.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/46001aee0695.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/46001aee0695.tar.gz` (if generated)